Reviewer Pack — Minimal Order of Quaternionic Kähler Forms Bounds Read-Twice…

Entry 462c76e5a32f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 08:22:54 UTC

Minimal Order of Quaternionic Kähler Forms Bounds Read-Twice BP Size

Entry ID: `462c76e5a32f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 08:22:54 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Geometry (Kähler Manifolds) **Field B** (complexity object): Boolean Function Complexity (Read-Twice Branching Program Size)

Statement:

> ['For a given n-bit boolean function f, the minimal order of a quaternionic Kähler form associated with f is at most O(n^2 log n), and for the trivial read-twice BP associated with the identity function, the minimal order is Ω(1).', "Equivalently, for all instances with property P (where P denotes an instance's boolean function being a trivial read-twice BP), property Q holds (where Q denotes the minimal order of a quaternionic Kähler form being at most O(n^2 log n))."]

Rationale (proposer's reasoning):

> ['Quaternionic Kähler forms provide a geometric structure that could potentially capture the complexity of boolean functions, akin to how symplectic structures relate to quantum mechanics. This conjecture aims to link this geometric object with the complexity of read-twice BP sizes.', 'If true, it would suggest a novel way to approach circuit lower bounds by using quaternionic geometry.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3341e841145b9206`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given n-bit boolean function f, if the minimal order of its associated quaternionic Kähler form is less than or equal to O(n^2 log n) for all generated functions and Spearman's rank correlation coefficient between minimal order and read-twice BP size exceeds 0.7, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quaternionic Kähler forms" AND "Boolean function complexity" AND read-twice BP - "Minimal order" OF "quaternionic Kähler forms" AND "boolean function complexity" AND size of BP - "Read-Twice BP" associated with identity function AND minimal order of quaternionic Kähler form ≤ O(n^2 log n)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_quaternionic_kähler_form(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Input must be a valid n-bit boolean function")

        # Simplified computation of the quaternionic Kähler form
        return sum(f[i] * i for i in range(2**n))

    def read_twice_bp_size(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Input must be a valid n-bit boolean function")

        # Simplified computation of the read-twice BP size
        return n * (n + 1)

    def spearman_rank_correlation(x, y):
        if len(x) != len(y):
            raise ValueError("x and y must have the same length")
```

```
        n = len(x)
        sorted_x = sorted(range(n), key=lambda i: x[i])
        sorted_y = sorted(range(n), key=lambda i: y[i])

        rank_x = [sorted_x.index(i) for i in range(n)]
        rank_y = [sorted_y.index(i) for i in range(n)]

        n = len(x)
        sum_diff_squares = sum((rank_x[i] - rank_y[i]) ** 2 for i in range(n))

        return 1 - (6 * sum_diff_squares) / (n * (n**2 - 1))

    results = []
    for _ in range(30):
        n = random.choice([10, 20, 30])
        f = generate_random_boolean_function(n)

        kähler_form_order = compute_quaternionic_kähler_form(f)
        bp_size = read_twice_bp_size(f)

        results.append((kähler_form_order, bp_size))

    if not results:
        return {
            "metric_name": "Spearman's rank correlation coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No instances generated"
        }

    kähler_form_orders, bp_sizes = zip(*results)
    correlation_coefficient = spearman_rank_correlation(kähler_form_orders, bp_sizes)

    return {
        "metric_name": "Spearman's rank correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "conjecture_holds": correlation_coefficient > 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(re
        support_fraction = 1.0
```

```
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any("conjecture_holds" in r and not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if "conjecture_hold
        print(f"RESULT: FALSIFIED counterexample=\"Spearman's rank correlation coefficient < 0.7\"
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means we cannot confirm or refute the conjecture based on the pre-registered support and falsification conditions. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12055
2	preregistration	ollama_remote	glm4:latest	0	0	5805
3	novelty	ollama_remote	glm4:latest	0	0	5025
4	novelty	ollama_remote	glm4:latest	0	0	5588
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32554
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10109
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13146
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10779
9	judge	ollama_remote	glm4:latest	0	0	21541

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 116602 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/462c76e5a32f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/462c76e5a32f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/462c76e5a32f.tar.gz` (if generated)